

Informe Hash

Tomás Jaures

Programación Avanzada

ICC708-1



Explicación

Una tabla hash es una estructura de datos diseñada para realizar operaciones de búsqueda, inserción y eliminación con una complejidad promedio de $O(1)$. Su principal ventaja es evitar búsquedas secuenciales, permitiendo un acceso rápido y eficiente a la información en comparación con otras estructuras de datos como arreglos o listas enlazadas.

El funcionamiento de una tabla hash se basa en el uso de una función hash, la cual transforma una clave en un valor numérico utilizado como índice dentro de la tabla. Este índice determina la posición en la que se almacenará el elemento. Dependiendo de la calidad de la función hash y del factor de carga de la tabla, las colisiones pueden disminuir o aumentar considerablemente.

Repositorio

Código con tabla hash implementada:

<https://github.com/TomasJaures/Talleres>

Tabla Comparativa

Dataset: RANDOM

Propiedad	Sum	Polynomial
Size	211	211
Elements	1000	1000
Load Factor	4,739	4,739
Collisions	886	790
Used Buckets	114	210
Max Bucket Size	27	12

Insert Time	0,002538	0,000753
-------------	----------	----------

Terminal

```
Dataset: random
strategy=sum, size=211, elements=1000, loadFactor=4,739, collisions=886, usedBuckets=114, maxBucketSize=27, insertTimeSeconds=0,002538
strategy=polynomial, size=211, elements=1000, loadFactor=4,739, collisions=790, usedBuckets=210, maxBucketSize=12, insertTimeSeconds=0,000753
```

Dataset: SEQUENTIAL

Propiedad	Sum	Polynomial
Size	211	211
Elements	1000	1000
Load Factor	4,739	4,739
Collisions	945	789
Used Buckets	55	211
Max Bucket Size	70	8
Insert Time	0,000514	0,000247

Terminal

```
Dataset: random
strategy=sum, size=211, elements=1000, loadFactor=4,739, collisions=886, usedBuckets=114, maxBucketSize=27, insertTimeSeconds=0,002538
strategy=polynomial, size=211, elements=1000, loadFactor=4,739, collisions=790, usedBuckets=210, maxBucketSize=12, insertTimeSeconds=0,000753
```

Dataset: CLUSTERED

Propiedad	Sum	Polynomial
Size	211	211
Elements	1000	1000
Load Factor	4,739	4,739
Collisions	945	789
Used Buckets	55	211
Max Bucket Size	70	10
Insert Time	0,000586	0,000297

Terminal

```
Dataset: clustered
strategy=sum, size=211, elements=1000, loadFactor=4,739, collisions=945, usedBuckets=55, maxBucketSize=70, insertTimeSeconds=0,000586
strategy=polynomial, size=211, elements=1000, loadFactor=4,739, collisions=789, usedBuckets=211, maxBucketSize=10, insertTimeSeconds=0,000297
```

Comparación completa:

Dataset	Propiedad	Estrategia Sum	Estrategia Polynomial
RANDOM	Size	211	211
	Elements	1000	1000
	Load Factor	4,739	4,739
	Collisions	886	790
	Used Buckets	114	210

	Max Bucket Size	27	12
	Insert Time (s)	0,002538	0,000753
---	---	---	---
SEQUENTIAL	Size	211	211
	Elements	1000	1000
	Load Factor	4,739	4,739
	Collisions	945	789
	Used Buckets	55	211
	Max Bucket Size	70	8
	Insert Time (s)	0,000514	0,000247
---	---	---	---
CLUSTERED	Size	211	211
	Elements	1000	1000
	Load Factor	4,739	4,739
	Collisions	945	789
	Used Buckets	55	211
	Max Bucket Size	70	10
	Insert Time (s)	0,000586	0,000297

Discusión

La función hash que produjo menos colisiones en los tres datasets fue la función polynomial. Esto se debe a que considera el orden de los caracteres y genera una distribución más uniforme de los elementos dentro de la tabla.

Los datasets SEQUENTIAL y CLUSTERED presentaron resultados muy similares entre sí, ya que sus claves poseen patrones repetitivos. Aun así, la estrategia polynomial logró

mantener una distribución eficiente y reducir considerablemente el tamaño máximo de los buckets.

Conclusión

En conclusión, la tabla hash es una estructura de datos extremadamente eficiente y ampliamente utilizada debido a que permite realizar operaciones de búsqueda, inserción y eliminación en un tiempo promedio de $O(1)$. Su rendimiento depende principalmente de la función hash utilizada, ya que esta determina cómo se distribuyen los elementos dentro de la tabla. Durante el desarrollo del laboratorio se observó que estrategias simples, como la suma de caracteres, generan una mayor cantidad de colisiones y una distribución menos uniforme de los buckets. Por otro lado, funciones más avanzadas, como el hashing polinomial, logran aprovechar de mejor manera el espacio disponible y reducir significativamente los conflictos entre elementos. También se comprobó la importancia del factor de carga, debido a que una tabla demasiado llena incrementa las colisiones y disminuye el rendimiento general. Finalmente, técnicas como Separate Chaining permiten manejar colisiones de manera eficiente mediante el uso de buckets y listas enlazadas, manteniendo el correcto funcionamiento de la tabla incluso bajo altas cargas de datos.